

Final Project Presentation

CS3710-002: Full-Stack Web
Development
William Maddock
12/06/2024



Web App Description

Overview: The AccessClearance Hub is a web application designed to streamline the process of managing and verifying access clearance levels for employees or contractors within an organization. Core functionalities include user authentication, clearance request submission, approval workflows, and automated clearance expiration notifications.

Audience: The app is designed for organizational administrators, security personnel, and employees who need to manage or request access clearances efficiently.

Tech Stack: The application leverages Ruby on Rails for backend development, Bootstrap for responsive frontend design, and Docker for environment consistency and deployment.

User Story

Content

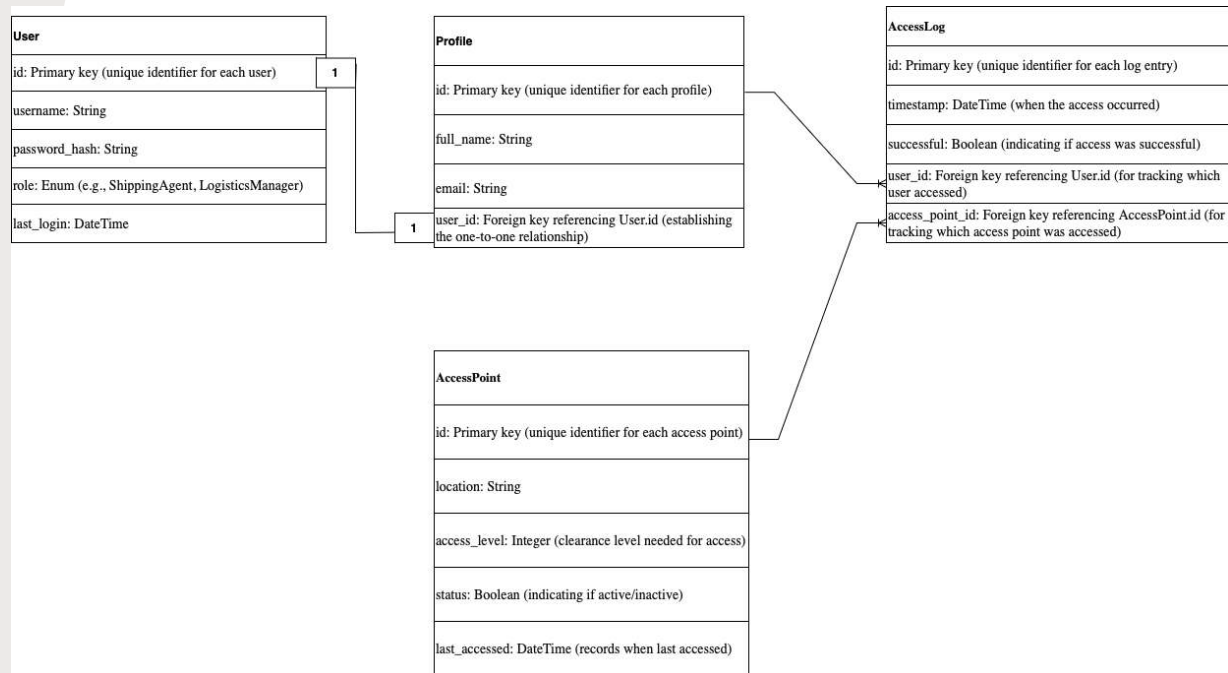
As a Logistics Manager, I want to approve or deny clearance requests so that I can ensure only authorized personnel gain access to sensitive areas

Example Scenarios

Scenario 1: A Shipping Agent submits a request for clearance to access a restricted lab. The Logistics Manager reviews the request, checks the Shipping Agent's credentials, and approves it.

Scenario 2: A Shipping Agent's clearance has become inactive. So, either the administrator or the Logistics Manager revoked the clearance.

Visuals: UML Diagram

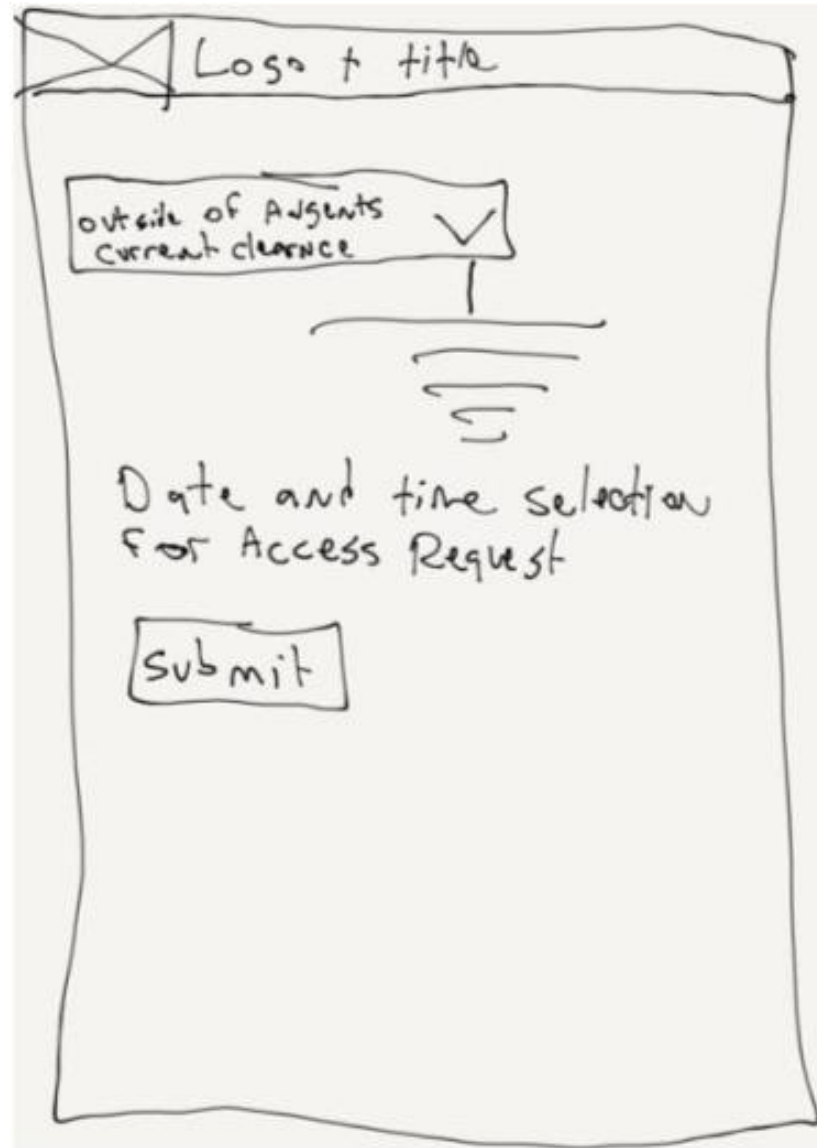


Lo-Fi Mockups

Mockup Screenshots: Include a wireframe of the “Create Access Request” page specifically designed for Shipping Agents. The wireframe should display a drop-down list for access points, a date and time selection widget, and a submit button.

Explanation: This mockup aligns with the user story by enabling Shipping Agents to request access to areas outside their current clearance. It supports the scenario where the agent needs temporary access, streamlining the approval process by forwarding the request to their supervisor.

Visuals: Wireframe for Access Request Page



Docker Setup

Purpose: Docker was used to ensure environment consistency across development, testing, and deployment. By containerizing the application, the setup process becomes streamlined, reducing the likelihood of issues due to differences in developer environments. It also simplifies deployment and provides an isolated workspace for running the application.

Setup: The Docker image was built using the following **command**:

```
docker buildx build -t  
bigwill12/msucs3710_name_it_whatever .
```

The application is run using the following **command** to map the local directory and ports:

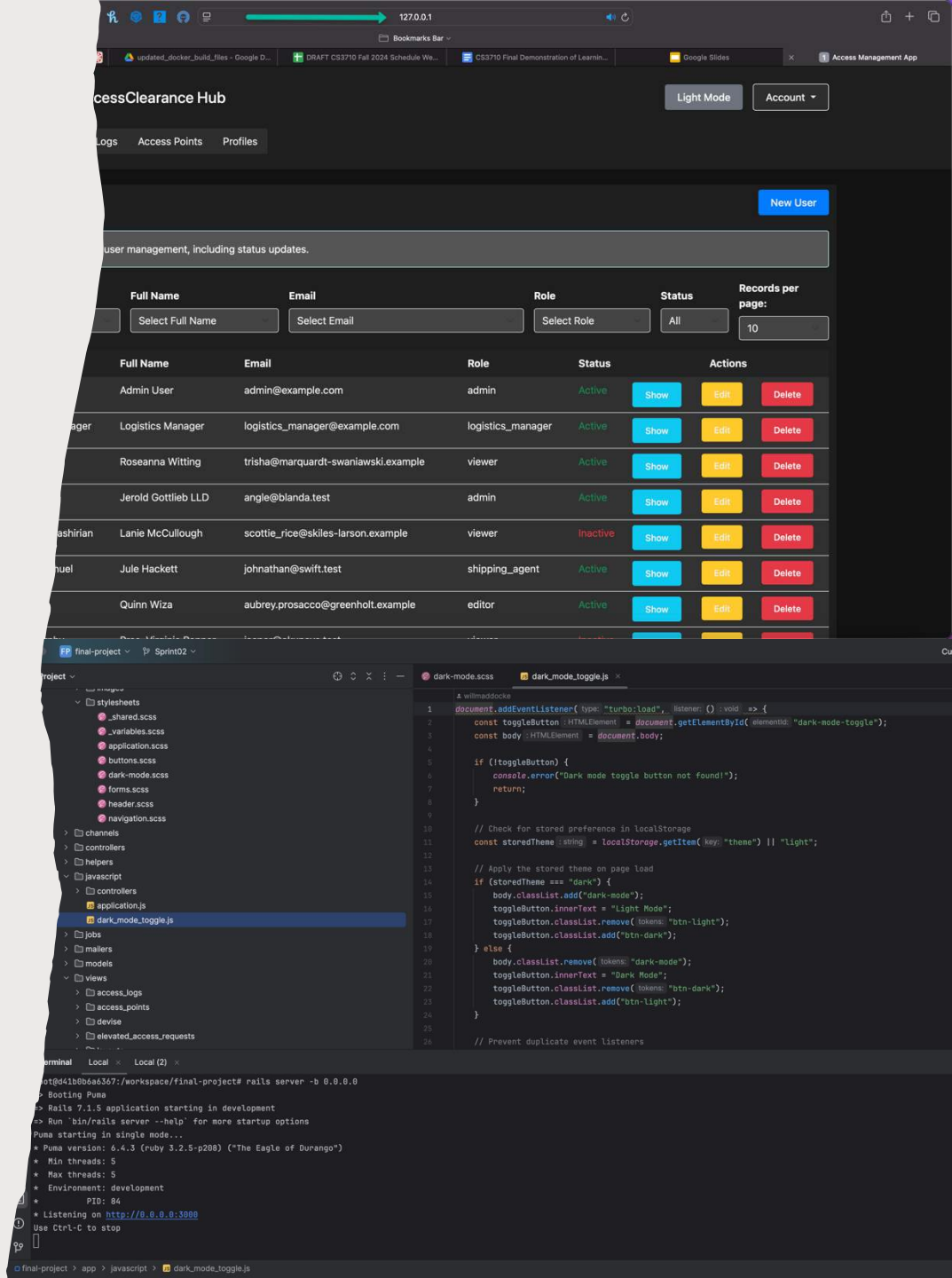
```
docker run -it -p 3000:3000 -v  
"$ (pwd) :/workspace"  
bigwill12/msucs3710_name_it_whatever
```

Inside the Docker container, the Rails server is started using this **command**:

```
rails server -b 0.0.0.0
```

Visuals: Screenshot of Docker in action.

Note: Docker and Ruby on Rails setup followed the instructions provided in the linked Google Drive resource: [Docker and Rails Setup](#).



Version Control

Purpose: Version control is essential for tracking changes and maintaining a reliable development history. It ensures that any mistakes can be reverted without losing progress.

Usage: The project is hosted on GitHub, where meaningful commit messages describe each change. Commit messages follow a clear format, such as:

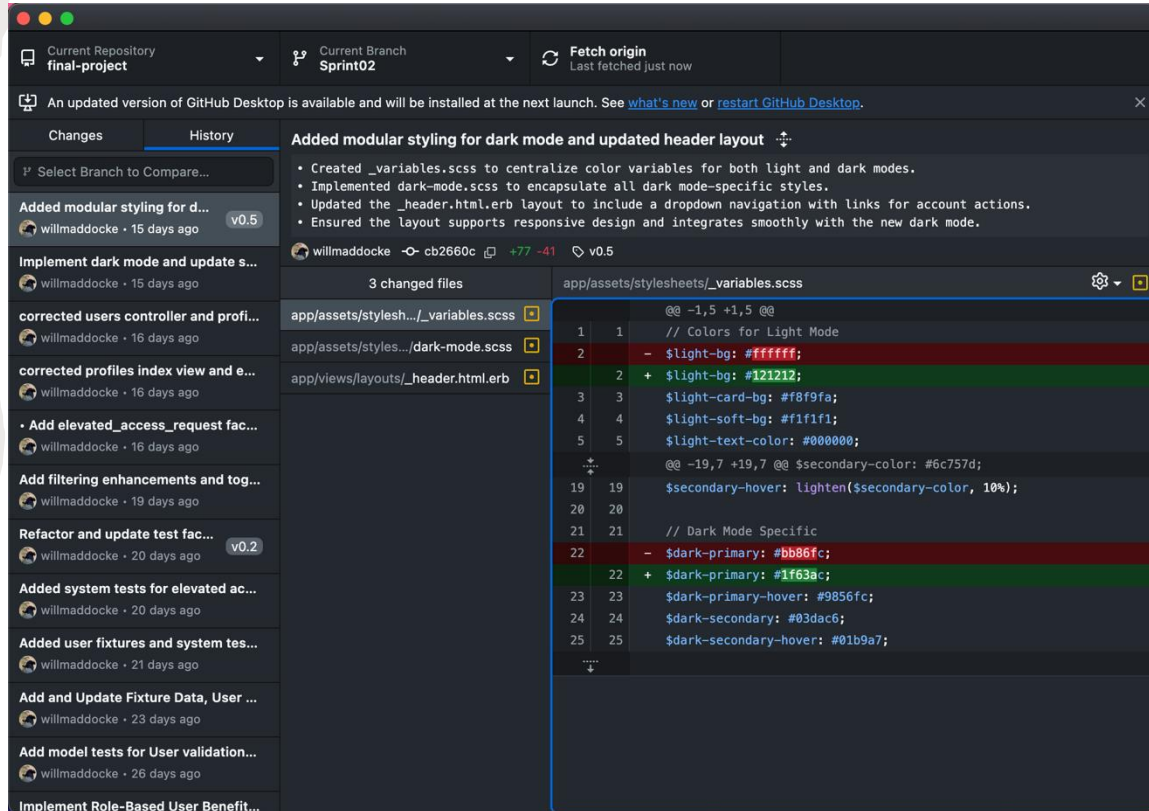
- “Add user authentication functionality”
- “Fix bug in access request form validation”

Workflow: The branching strategy used includes:

Main Branch: Stable version of the project, ready for deployment.

Sprint Branches: Separate branches for Sprint01 and Sprint02.

Pull Requests: Feature branches are reviewed and tested before merging into the main branch to ensure stability.



Testing with RSpec

BDD and TDD: The application follows both Behavior-Driven Development (BDD) and Test-Driven Development (TDD). Using RSpec, BDD is applied by writing system tests that simulate user interactions with the web application, ensuring the behavior aligns with user stories. TDD is demonstrated by creating model tests to validate business logic and application integrity before implementing features.

Examples:

1. System Test:

The `spec/system/restricted_area_access_spec.rb` file includes tests for:
Requesting elevated access and receiving approval or denial.
Viewing elevated access request details.
Handling pagination and filtering of requests.

Example 1:

```
scenario "Shipping agent requests access and gets approved" do
  visit root_path
  # Test steps...
  expect(page).to have_text "Approved"
end
```

This test validates end-to-end workflows, ensuring that roles interact with the system as intended.

2. Model Test:

The `spec/models/elevated_access_request_spec.rb` file validates the `ElevatedAccessRequest` model for required attributes such as `user`, `access_point`, and `reason`.

Example 2:

```
context "without an access point" do
  it "is not valid" do
    elevated_access_request.access_point = nil
    expect(elevated_access_request).not_to be_valid
  end
end
```

```
williammaddock — docker exec -it stoic_kowalevski /bin/bash
root@d41b0b6a6367:/workspace/final-project# rspec spec/system/restricted_area_access_spec.rb --format documentation
DEPRECATION WARNING: `Rails.application.secrets` is deprecated in favor of `Rails.application.credentials` and will be removed in Rails 7.2. (called from <top (required)> at /workspace/final-project/config/environment.rb:5)

Restricted Area Access
  Shipping agent requests access and gets approved
  Shipping agent requests access and gets denied
  View elevated access request details
  Pagination and filtering elevated access requests

Finished in 3.96 seconds (files took 1.15 seconds to load)
4 examples, 0 failures

root@d41b0b6a6367:/workspace/final-project#
```

```
williammaddock — docker exec -it stoic_kowalevski /bin/bash — 80x
Last login: Wed Nov 20 13:53:24 on ttys000
williammaddock@MacBook-Pro ~ % docker exec -it stoic_kowalevski /bin/bash
root@d41b0b6a6367:/workspace# ls
Dockerfile  Gemfile  Gemfile.lock  final-project
root@d41b0b6a6367:/workspace# cd final-project
root@d41b0b6a6367:/workspace/final-project# rspec spec/models/elevated_access_request_spec.rb --format documentation
DEPRECATION WARNING: `Rails.application.secrets` is deprecated in favor of `Rails.application.credentials` and will be removed in Rails 7.2. (called from <top (required)> at /workspace/final-project/config/environment.rb:5)

ElevatedAccessRequest
  with valid attributes
    is valid
  without a user
    is not valid
  without an access point
    is not valid

Finished in 0.22071 seconds (files took 1.1 seconds to load)
3 examples, 0 failures

root@d41b0b6a6367:/workspace/final-project#
```


HTML/CSS

Purpose: The user interface for the web stack demonstrates the foundational design and responsiveness of the application, showcasing basic but functional styling. The design ensures clear navigation and intuitive interaction with the application's core features while adhering to industry standards for web development.

Examples:

1. Navigation Bar:

Simple navigation bar styled for clarity, featuring links to key areas such as "Home," "Access Requests," and "Account."

Example: Bootstrap's navbar component is used for responsive navigation across different screen sizes.

2. Forms:

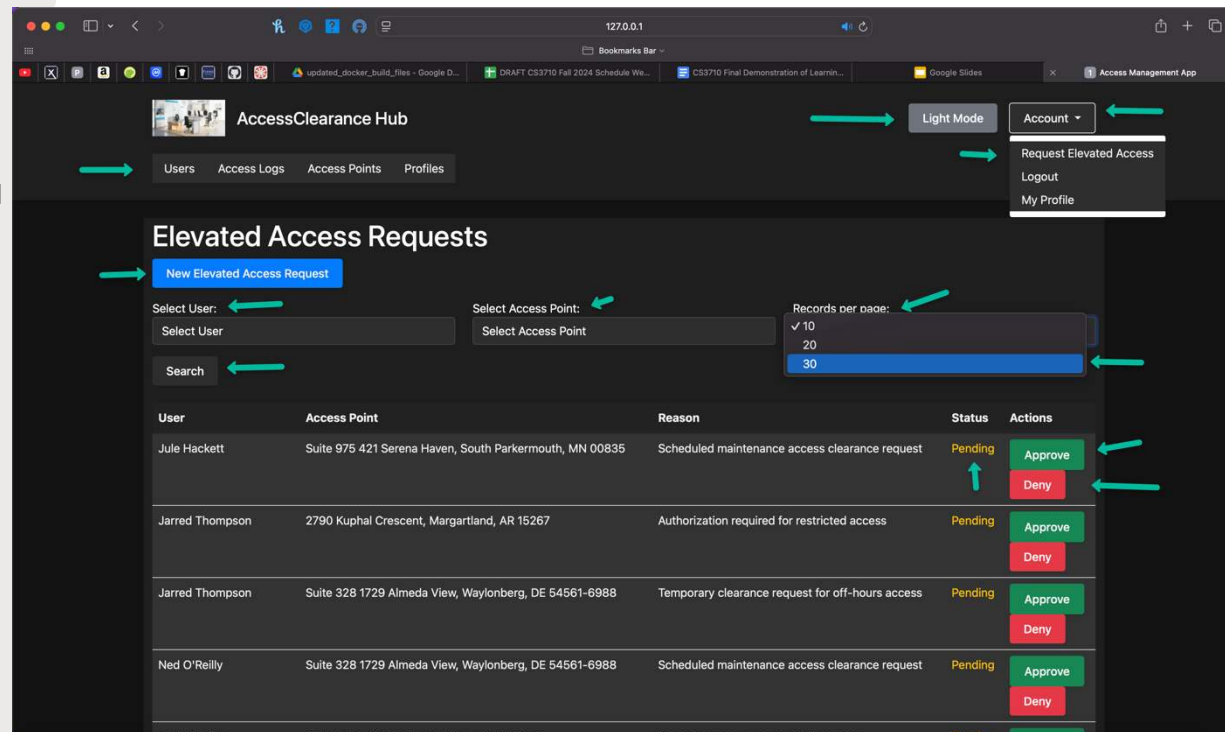
Access request forms styled with Bootstrap's form-control and btn classes to ensure a consistent and user-friendly look.

Example: Dropdown menus for access points and date pickers for request scheduling are styled for ease of use.

3. Layouts:

The layout is built with Bootstrap's grid system, ensuring that the application adapts to various screen sizes, providing a responsive and clean user experience.

Example: Single-column layout for mobile devices and multi-column layouts for desktop views.



Accessibility

Principles: The application follows the Web Content Accessibility Guidelines (WCAG) to ensure inclusivity and usability for all users, including those with disabilities. Key principles include:

Perceivable: Ensuring all information is presented in ways users can perceive (e.g., text alternatives for non-text content).

Operable: Designing interfaces usable by keyboard and assistive technologies.

Understandable: Creating predictable navigation and error handling.

Robust: Ensuring compatibility with a wide range of user agents, including assistive technologies.

Examples:

1. **ARIA Labels:**

Added ARIA labels to form elements for users relying on screen readers.

Example: Buttons and dropdowns include descriptive labels such as `aria-label="Submit Access Request"`.

2. **Keyboard Navigation:**

Ensured all interactive elements (e.g., links, buttons, and form fields) are accessible via keyboard navigation (e.g., Tab and Enter keys).

Focus indicators are visible to guide users through the interface.

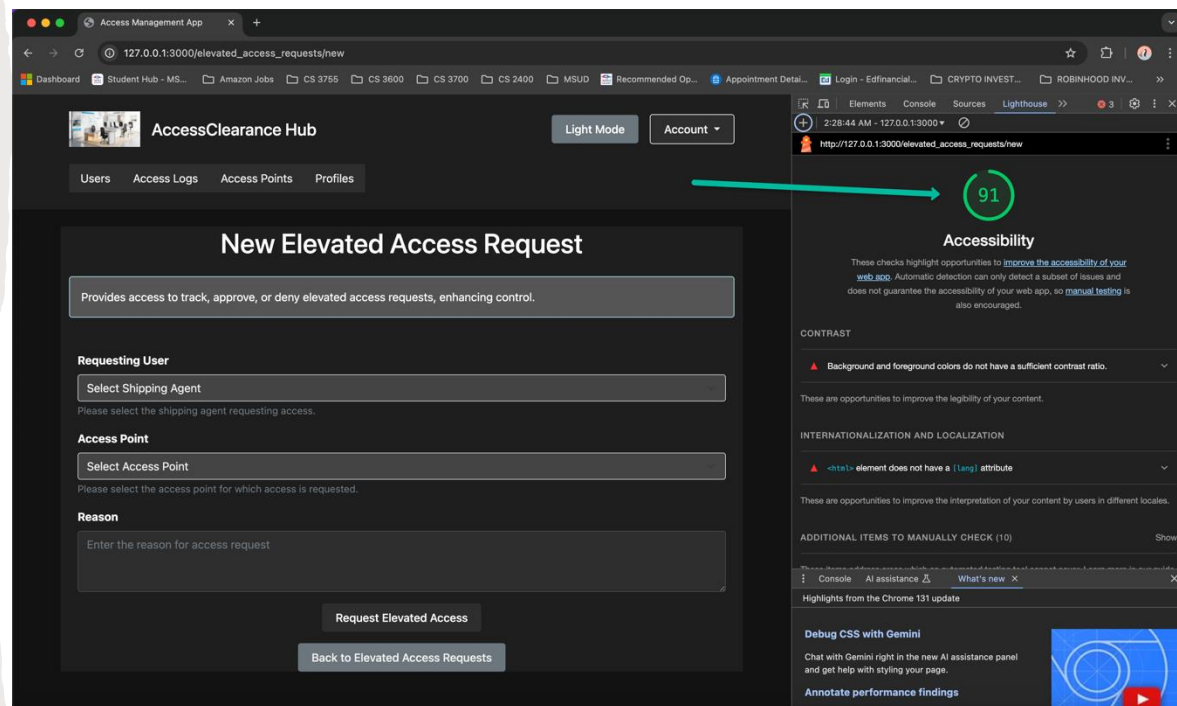
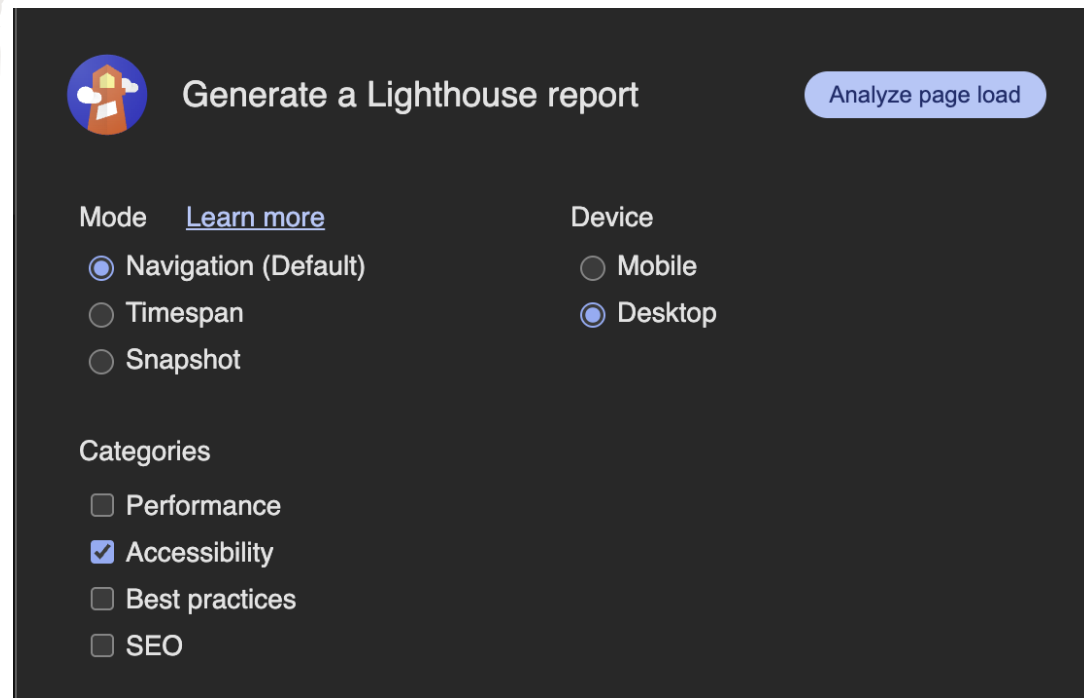
3. **Color Contrast:**

Used Bootstrap's accessible color palette to maintain sufficient contrast ratios for text and UI components.

Example: Buttons and text meet the contrast requirements for readability.

4. **Error Feedback:**

Clear error messages provided in forms, with alerts accessible via screen readers.



Security

Principles: The application incorporates essential security measures to protect user data and ensure a secure user experience:

User Authentication: Implemented using the devise gem for secure login, registration, and session management.

Data Validation: Enforced model validations to prevent invalid or malicious data input.

Role-Based Access Control (RBAC): Defined user roles (admin, viewer, shipping_agent, etc.) to restrict permissions and safeguard sensitive operations.

Examples:

1. Authentication with Devise:

Code Snippet (from app/models/user.rb):

```
devise :database_authenticatable, :registerable,  
      :recoverable, :rememberable, :validatable
```

This configuration ensures secure password handling, account recovery, and session persistence.

Registration Form (from app/views/devise/registrations/new.html.erb):

The form incorporates proper validations and secure password entry fields:

```
<%= f.password_field :password, autocomplete: "new-password", class: "form-control" %>
```

```
<%= f.password_field :password_confirmation, autocomplete: "new-password",  
class: "form-control" %>
```

2. Role-Based Access Control (RBAC):

User roles are defined using enum in the User model, ensuring controlled access to various functionalities:

```
enum role: { admin: 0, editor: 1, viewer: 2, shipping_agent: 3, logistics_manager: 4  
}
```

Custom methods like `can_request_access?` and `can_edit_items?` enable granular permission checks, protecting sensitive actions.

3. Data Validation:

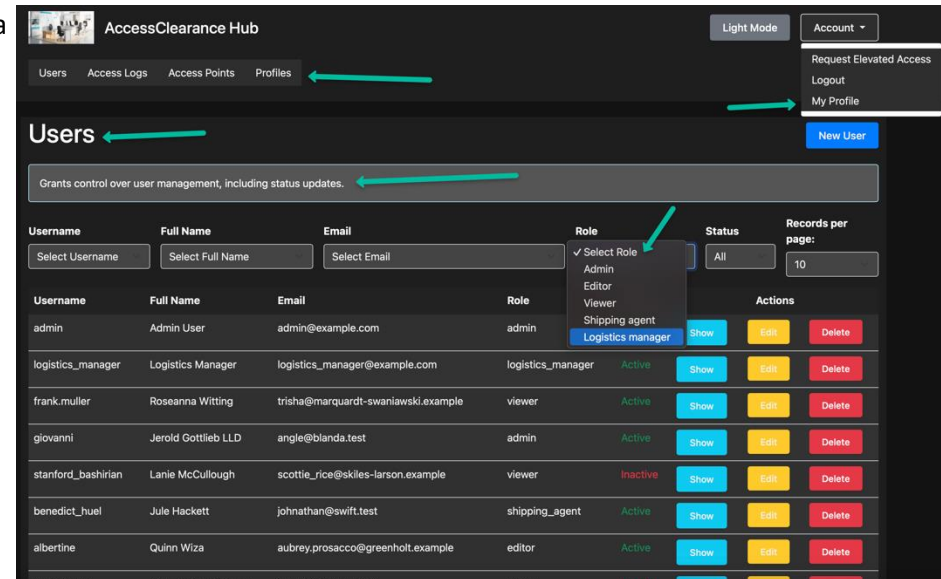
The User model enforces strict validation of critical fields:

```
validates :email, presence: true, uniqueness: true
```

```
validates :username, presence: true, uniqueness: true
```

```
validates :role, inclusion: { in: roles.keys }
```

This prevents duplicate records, invalid emails, or unauthorized role assignments.



Model-View-Controller

Explanation:

The Model-View-Controller (MVC) architecture organizes application logic into three interconnected components:

1. **Model:** Manages the data, logic, and rules of the application. In this application:

User: Represents users with attributes and methods for authentication and permissions.

Profile: Stores detailed user information such as personal and professional data.

ElevatedAccessRequest: Handles access request data, statuses, and associations with users and access points.

2. **View:** Presents data to the user and collects input. In this application:

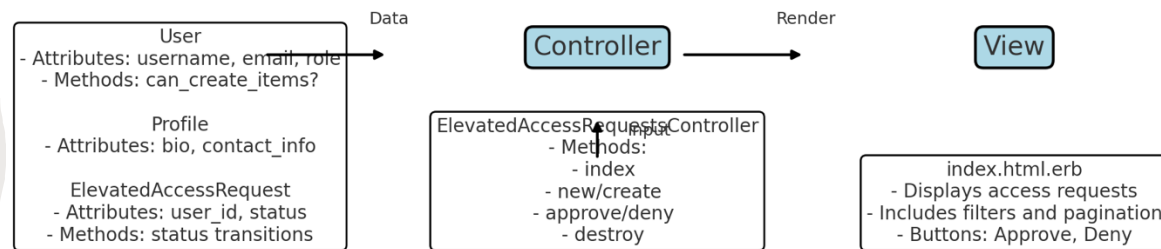
index.html.erb: Displays a list of elevated access requests, forms for filtering/searching, and actions like approve/deny.

3. **Controller:** Processes user inputs, interacts with the model, and selects the view to render. In this application:

ElevatedAccessRequestsController: Manages CRUD operations for elevated access requests, ensures user permissions, and handles approvals/denials.

This separation improves code modularity, making the application easier to maintain and scale. For instance, the `ElevatedAccessRequestsController` processes user actions (e.g., approve/deny), while the `ElevatedAccessRequest` model ensures data integrity, and the `index.html.erb` view ensures that users only see what they need.

Diagram: Custom diagram showing interactions between Model, View, and Controller.



Database Overview

Purpose: The database schema serves as the backbone of the application, organizing and structuring data to support functionality. It facilitates secure access management, user profiles, and record tracking. The key components of the schema include:

Users: Stores user information such as credentials, roles, and access levels, enabling role-based permissions and authentication.

Profiles: Extends user data with personal details like bio and avatar.

Elevated Access Requests: Tracks requests for access to restricted areas, associating each request with a user and an access point. Includes status management for approval workflows.

Access Points: Represents physical or virtual locations with an associated access level, aiding in access control logic.

Access Logs: Records user access attempts with timestamps and success status, enabling audit trails and monitoring.

These tables are interconnected through foreign key relationships, ensuring data integrity and enforcing referential constraints.

Diagram: Entity-Relationship Diagram (ERD) or schema screenshot.

```
88 create_table "users", force: :cascade do |t|
89   t.string "username", null: false
90   t.string "full_name"
91   t.string "email", null: false
92   t.string "encrypted_password", null: false
93   t.integer "role", default: 0, null: false
94   t.integer "access_level"
95   t.datetime "last_login"
96   t.boolean "status", default: true
97   t.datetime "created_at", null: false
98   t.datetime "updated_at", null: false
99   t.string "reset_password_token"
100  t.datetime "reset_password_sent_at"
101  t.datetime "remember_created_at"
102  t.boolean "active", default: true
103  t.index column_name ["email"], **options { name: "index_users_on_email", unique: true }
104  t.index column_name ["reset_password_token"], **options { name: "index_users_on_reset_password_token", unique: true }
105  t.index column_name ["role"], name: "index_users_on_role"
106  t.index column_name ["username"], name: "index_users_on_username"
107 end
108
109 add_foreign_key from_table "access_logs", to_table "access_points"
110 add_foreign_key from_table "access_logs", to_table "users"
111 add_foreign_key from_table "active_storage_attachments", to_table "active_storage_blobs", column: "blob_id"
112 add_foreign_key from_table "active_storage_variant_records", to_table "active_storage_blobs", column: "blob_id"
113 add_foreign_key from_table "elevated_access_requests", to_table "access_points"
114 add_foreign_key from_table "elevated_access_requests", to_table "users"
115 add_foreign_key from_table "profiles", to_table "users"
116 end
```

Object and Database Relationship

Explanation:

In Rails, Object-Relational Mapping (ORM) is managed through Active Record, which connects Ruby objects to database tables. Each model represents a table, and attributes correspond to columns in that table. Relationships between objects are defined using methods like `belongs_to`, `has_many`, and `has_one`, enabling complex interactions.

For example, in our application:

The User model represents the users table.

The ElevatedAccessRequest model represents the elevated_access_requests table.

Relationships like “a user has many elevated access requests” are defined in the model files, bridging the database and object layers.

Visuals: Code snippet showing a model's relationship definitions.

Code Example:

The following snippet demonstrates ORM relationships and definitions for User and ElevatedAccessRequest models.

User Model (app/models/user.rb):

```
class User < ApplicationRecord
  # Associations
  has_many :elevated_access_requests
  has_one :profile

  # Enum for roles
  enum role: { standard: 0, logistics_manager: 1, shipping_agent: 2 }

  # Validations
  validates :username, presence: true, uniqueness: true
  validates :email, presence: true, uniqueness: true
end
```

ElevatedAccessRequest Model (app/models/elevated_access_request.rb):

```
class ElevatedAccessRequest < ApplicationRecord
  # Associations
  belongs_to :user
  belongs_to :access_point

  # Scopes for status filtering
  scope :pending, -> { where(status: 'pending') }
  scope :approved, -> { where(status: 'approved') }
  scope :denied, -> { where(status: 'denied') }

  # Validations
  validates :reason, presence: true
end
```

Relationship Mapping:

1. Database Relationship:

users table has a one-to-many relationship with the elevated_access_requests table.

elevated_access_requests connects to access_points via foreign keys.

These relationships are enforced with `t.references` and `add_foreign_key` in migrations.

2. Object Interaction:

A User object can fetch its related elevated access requests with `user.elevated_access_requests`.

ElevatedAccessRequest retrieves its associated User with `request.user`.

Visual Example:

Here's a database-to-object relationship table:

Database Table	Active Record Association	Direction
users	has_many :elevated_access_requests	One-to-Many
elevated_access_requests	belongs_to :user	Many-to-One
access_points	has_many :elevated_access_requests	One-to-Many (optional)

This ORM system abstracts SQL queries, allowing the application to interact with data through intuitive object-oriented methods.

Reflection

Learning Outcomes: Throughout the development of this application, I gained significant experience in several key areas of software development:

1. **Full-Stack Development:**

I worked on both the front-end and back-end of the application. This included setting up models, migrations, and controllers for the database, as well as implementing user interfaces with HTML, CSS, and JavaScript.

I also utilized Rails as a backend framework, including its Active Record ORM to manage data, and incorporated Devise for user authentication.

2. **Database Management:**

I developed and maintained an efficient database schema to support the app's features, with a focus on relationships between models, data integrity, and optimization.

I gained proficiency in writing migrations and managing relationships using Active Record associations like `has_many`, `belongs_to`, and `has_one`.

3. **Testing:**

I gained experience in writing and running tests using tools like RSpec and Capybara to ensure the application functions correctly.

I also applied Test-Driven Development (TDD) principles to ensure that new features were implemented with a high level of code reliability.

4. **Agile Development:**

The development process was iterative, with frequent feedback loops that allowed for continuous improvement. I practiced agile methodologies, using tools like Trello to manage tasks and user stories.

5. **Security Practices:**

Implementing secure user authentication with Devise taught me the importance of password encryption, session management, and other security practices such as protection against SQL injection and cross-site scripting (XSS).

Future Improvements:

1. **Real-Time Notifications:**

Implementing a notification system that alerts users about changes to their access requests (approved, denied, etc.) in real-time using technologies like ActionCable or WebSockets.

2. **Role-Based Dashboards:**

Developing more customized dashboards based on user roles (e.g., administrators, logistics managers, standard users) would enhance the user experience and provide more focused access control.

3. **Improved Analytics and Reporting:**

Adding more sophisticated reporting features, such as generating reports on user access history, request statuses, and frequent access points, would provide more insights for administrators and help in decision-making.

4. **Mobile Optimization:**

Optimizing the user interface for mobile devices could enhance usability, particularly for users who need to access the application on the go. Using frameworks like Bootstrap or Tailwind CSS could aid in achieving this.

5. **Multi-Factor Authentication (MFA):**

Adding multi-factor authentication to improve security for users, particularly for those with elevated access levels, would be a valuable addition to the app.

By reflecting on these experiences and challenges, I can continue to evolve as a developer and refine the features and user experience of the app for future iterations.

Challenges and Solutions:

1. **Challenge:** Managing complex relationships between tables, such as users and access requests, was initially difficult, especially in terms of ensuring data integrity and proper foreign key constraints.

Solution: I took time to understand how Active Record associations and foreign key constraints work. I then refactored and structured the migrations to ensure that the relationships were properly defined and maintained across the app. Additionally, I used tools like rails db:rollback to iteratively test and adjust the schema.

2. **Challenge:** Implementing user authentication with Devise posed a challenge, particularly with configuring password resets and managing user sessions securely.

Solution: I closely followed Devise documentation and used online forums and tutorials to troubleshoot specific issues. I also wrote tests to ensure that the authentication system worked seamlessly, which helped pinpoint any bugs or configuration issues.

3. **Challenge:** Implementing real-time access log tracking for users was more complex than expected due to managing timestamps and frequent updates.

Solution: I solved this by breaking down the process into smaller steps and using background jobs to handle logging asynchronously. I also utilized database indexes to improve the speed of read operations on log data.

4. **Challenge:** Ensuring the application remained responsive and user-friendly, especially with dynamic forms for access requests.

Solution: I used JavaScript and AJAX to improve user experience by asynchronously submitting forms and updating UI elements without needing full-page reloads.